

UNIVERSIDADE FEDERAL DO RIO GRANDE DO SUL
INSTITUTO DE INFORMÁTICA
DEPARTAMENTO DE INFORMÁTICA APLICADA
INF01154 - Redes de Computadores N

Nível de aplicação: HTTP, DNS, Programação Ping (Parte 1).

1 Animação: HTTP delay estimation

Apresentar em aula a animação referente à estimativa de atraso no HTTP. Acesso em https://media.pearsoncmg.com/aw/ecs_kurose_compnetwork_7/cw/content/interactiveanimations/http-delay-estimation/index.html.

- a) Escolher “non-persistent connections” com 4 objetos e atraso de transmissão para cada objeto de 1 RTT. Comentar.
- b) Escolher “non-persistent connections with paralellism” com 2 conexões em paralelo. O restante mantém. Comentar.
- c) Escolher “persistent connections without pipeline”. O restante mantém. Comentar.
- d) Escolher “persistent connections with pipeline”. O restante mantém. Comentar.

2 Wireshark: HTTP

Quem tiver curiosidade, assistir o vídeo do Jim Kurose para wireshark e HTTP: https://mediaplayer.pearsoncmg.com/_ph_cc_ecs_set.title.Introduction_to_Wireshark_and_HTTP_/aw/streaming/ecs_kurose_compnetw_6/wireshark_videonotes.m4v

Abra o arquivo do laboratório de wireshark HTTP do Kurose, disponível na página da disciplina (*Cap2_Wireshark1_HTTP_v7.0.pdf*). **Execute todos os cinco exercícios de forma online**, ou seja, utilizando as URLs fornecidas no arquivo. Teste no navegador o uso de “F5” e “SHIFT-F5”. Algumas vezes será útil.

- a) No item 1 (*Basic HTTP GET/response*), observe que haverá um segundo GET para o arquivo “favicon.ico”, que serve para mostrar um ícone ao lado da URL. Esse pode ser descartado.
- b) No item 2 (*HTTP Conditional GET/response*), se você estiver usando Chrome, a limpeza de cache é feita em “☰ + Configurações + Privacidade e Segurança + Limpar dados de navegação”.
- c) No item 3 intitulado (*Retrieving Long Documents*), uma possibilidade de filtro para facilitar a visualização é: (*http or tcp*) and *ip.addr==128.119.245.12* and *tcp.port==62207*. Verifique a porta exata da conexão.
- d) Uma forma gráfica de seguir um determinado fluxo no *wireshark* é utilizar “*statistics + flow-graph*”. Escolha a opção de “*displayed packets*” após criar o filtro de visualização do item anterior.
- e) **Curiosidade:** algumas vezes pode-se ver o tamanho do pacote no *wireshark* (sem CRC) sendo maior que o máximo (1514 bytes num quadro Ethernet). Isso acontece porque a *libpcap* do *wireshark* captura os pacotes ANTES do mesmo ir para a placa de rede. Existe uma opção que o SO envia pacotes maiores para serem fragmentados na placa. O nome é: *Large Receive Offload (LRO) or Receive Segment Coalescing (RSC)* quando se recebem pacotes maiores, e *TCP Segmentation Offload (TSO) aka Large Segment Offload (LSO) aka Generic Segment Offload (GSO)* quando se transmitem pacotes maiores. Um artigo explicando isso pode ser visto em: <http://packetbomb.com/how-can-the-packet-size-be-greater-than-the-mtu/>. Na verdade os pacotes **não são maiores**. Eles só não foram fragmentados ainda pela placa de rede.

- f) **Dica:** caso você queira analisar com maiores detalhes cabeçalhos HTTP e outras singularidades, pode utilizar o ambiente de desenvolvimento de APIs chamado “Postman” (<https://www.getpostman.com>).

3 PROGRAMAÇÃO: Web Server

Abra o arquivo do programa de *web server* do Kurose, disponível na página da disciplina (*Cap2_Prog1_WebServer.pdf*). Copie o código em *python*, adicionando ao mesmo as linhas faltantes (*#Fill in start* e *#Fill in end*). Se quiser utilizar outra linguagem, OK. Seu código deve:

- a) Abrir um arquivo html (*index.html* ou *helloworld.html* ou outro) a partir de um navegador Internet (usando a URL, por exemplo, *localhost:6789/index.html*).
- b) Dicas: basicamente estão faltando as primitivas de sockets, que são para TCP:
 1. *Bind* e *Listen* (amarra o IP com a porta e determina fila de aceite de conexões)
 2. *Accept* (o servidor se prepara para receber a conexão)
 3. *Receive* (recebe o pedido HTTP GET dentro do laço)
 4. *Read* do arquivo solicitado (*f.read()*)
 5. *Send* da mensagem de resposta (200 OK)
 6. No caso de *IOError* (arquivo não encontrado), falta o *Send* de “*Not Found*” e fechar o socket (*Close*).
- c) Para executar o servidor a partir de um terminal Linux, pode-se utilizar a linha de comando: “*python3 nomearq.py*”.